M4 Math Note

Chung-En (Johnny) Yu
Department of Intelligent Systems and Robotics, University of West Florida

> **Tip**
>
> This math note serves as supplementary content for the lecture. Students are encouraged to read it in conjunction with the lecture.

Data Preprocessing

Overview

Preprocessing prepares raw data for machine learning models. Key steps often include:

- **Standardization and Normalization**: Transform data to a consistent scale for improved optimization.
- **Handling Missing Data**: Methods to fill or remove incomplete entries.
- **Encoding Categorical Variables**: Converting non-numeric data into a numeric format.
- **Feature Scaling and Selection**: Adjusting data scales and selecting relevant features.

Importance of preprocessing for improving model performance

The mathematical foundation for preprocessing is rooted in optimization. For instance:

1. **Gradient Descent Efficiency**: When features are scaled similarly, gradient descent converges faster because the optimization surface is more spherical than elongated. This minimizes issues like vanishing gradients.
2. **Mitigating Bias and Variance**: Preprocessing reduces noise, enhancing model generalization.

Dealing with Missing Data

Source: GeeksforGeeks
Source: datacamp

Overview

Missing data occurs when certain observations in the dataset are incomplete. It is categorized into:

1. **Missing Completely at Random (MCAR)**: Missingness is independent of any data or feature.

    - This occurs when every variable and observation has an equal chance of being missing. Imagine a child playing with Lego pieces of various colors to build a house. If some pieces are lost during the game, it represents missing information. The loss is random and does not affect the information about the remaining pieces.
2. **Missing at Random (MAR)**: Missingness depends on observed data.
    - In this case, the likelihood of a value being missing is related to the value of the variable or other variables in the dataset. For example, in a survey among data scientists, those who do not frequently update their skills might skip questions about new algorithms or technologies. The missing data is related to how often the data scientist updates their skills.
3. **Missing Not at Random (MNAR)**: Missingness depends on unobserved data.
    - This is the most challenging scenario, where the probability of missing data varies for different values of the same variable, and the reasons might be unknown. For instance, in a survey about married couples, those with troubled relationships might avoid answering certain questions due to embarrassment.

Techniques for handling missing data

1. Removing Missing Data

- Removing rows or columns with missing values assumes that the remaining data satisfies MCAR, ensuring no bias.
- Let $X \in \mathbb{R}^{m \times n}$ be the dataset. Removing missing entries transforms X into $X' \subseteq X$, where:

$$X' = \{x_{ij} \mid x_{ij} \neq \text{NaN}\}$$

2. Imputation Methods

- **Mean/Median Imputation**:
- Replace missing values with the mean μ or median of the observed values for that feature:

$$x_j = \frac{1}{|M|} \sum_{i \in M} x_{ij}, \quad \text{where } M = \{i \mid x_{ij} \neq \text{NaN}\}$$

- **Regression Imputation**:
- Use a regression model to predict missing values. Let X_{obs} be observed features and X_{miss} the missing ones. The regression model minimizes:

$$\min_{\theta} \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(X_{\text{obs}}^{(i)}) - X_{\text{miss}}^{(i)} \right)^2$$

- **K-Nearest Neighbors (KNN) Imputation**:
- Replace missing values using the average of the k-nearest samples. Given $X_{\text{miss}}^{(i)}$:

$$x_{ij} = \frac{1}{k} \sum_{p \in N_k(i)} x_{pj}, \quad N_k(i) \text{ is the set of } k \text{ nearest neighbors.}$$

3. Matrix Factorization-Based Imputation

- Use low-rank approximations to reconstruct missing data. Let X have missing entries. Decompose X into two matrices:

$$X \approx UV^T, \quad U \in \mathbb{R}^{m \times k}, V \in \mathbb{R}^{n \times k}$$

Solve:

$$\min_{U,V} \sum_{(i,j) \in \Omega} \left(x_{ij} - (UV^T)_{ij}\right)^2, \quad \Omega = \{(i,j) \mid x_{ij} \neq \text{NaN}\}$$

Pros and Cons

Method	Advantages	Disadvantages
Removing Missing Data	Simple to implement; makes no assumptions about the data.	Reduces dataset size; can introduce bias if data are not Missing Completely at Random (MCAR).
Mean/Median Imputation	Fast and easy to implement.	Ignores relationships between features; can underestimate variability.
Regression Imputation	Accounts for correlations between features; provides more accurate estimates.	Computationally intensive; assumes a linear relationship between variables.
KNN Imputation	Captures local data structure; can handle complex relationships.	Sensitive to outliers; performance depends on the choice of 'k' and distance metric.
Matrix Factorization Imputation	Preserves global data structure; suitable for high-dimensional data.	Complex to implement; sensitive to parameter tuning; may require large datasets.

Handling Categorical Data

Overview

Categorical data represents variables that contain label values rather than numeric values. These can be divided into:

1. **Nominal Data**: Categories without an inherent order (e.g., color: red, blue, green).
2. **Ordinal Data**: Categories with a meaningful order but no fixed interval between them (e.g., ratings: poor, average, good).

Encoding techniques

Since many machine learning algorithms require numerical input, it’s essential to convert categorical data into numerical formats. Common encoding techniques include:

1. **Label Encoding**
   - **Description**: Assigns a unique integer to each category.

- **Use Case**: This works only for ordinal categorical features (where order matters).
- **Example**:

Category	Encoded Value
Red	0
Green	1
Blue	2

2. **One-Hot Encoding**
   - **Description**: Creates binary columns for each category, indicating the presence (1) or absence (0) of the category.
   - **Use Case**: Ideal for nominal data without intrinsic order.
   - **Advantage**: No artificial ordering like Label Encoding.
   - **Disadvantage**: Can cause high-dimensionality if many unique categories exist.
   - **Example**:

Color	Red	Green	Blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1

3. **Target Encoding**
   - **Description**: Replaces categories with the mean of the target variable for each category.
   - **Advantage**: Works well with high-cardinality categorical features.
   - **Disadvantage**: Leads to data leakage if not handled correctly (e.g., should be used only on training data).
   - **Example**:
     If predicting house prices, and the "neighborhood" feature is categorical, replace each neighborhood with the average house price in that neighborhood.
4. **Frequency Encoding**
   - **Description**: Replaces categories with their frequency in the dataset.
   - **Use Case**: Helps capture the impact of rare or common categories.
   - **Example**:

Category	Frequency
Red	50
Green	30
Blue	20

5. **Binary Encoding**

- **Description**: Combines label and binary encoding by converting label indices to binary and then splitting the digits into separate columns.
- **Use Case**: Reduces dimensionality compared to one-hot encoding, especially with high-cardinality features.
- **Example**:

Category	Label	Binary Code	Col1	Col2	Col3
Red	1	001	0	0	1
Green	2	010	0	1	0
Blue	3	011	0	1	1

Choosing encoding techniques

- **Cardinality**: High-cardinality features (many unique categories) can lead to high-dimensional data with one-hot encoding. Alternative methods like target or frequency encoding may be more appropriate.
- **Ordinality**: For ordinal data, label encoding preserves the order, while one-hot encoding does not.
- **Algorithm Requirements**: Some algorithms (e.g., tree-based methods) can handle categorical data without explicit encoding, while others (e.g., linear regression) require numerical input.

Feature Scaling

Source: Geeksforgeeks

Overview

Feature scaling is a crucial preprocessing step in machine learning that involves adjusting the range of independent variables or features in your data. The goal is to ensure that each feature contributes equally to the model, preventing features with larger magnitudes from disproportionately influencing the outcome. This is particularly important for algorithms sensitive to the scale of data, such as those relying on distance metrics or gradient descent optimization. Decision trees and random forests are two of the very few machine learning algorithms where we don't need to worry about feature scaling.

Why feature scaling is important?

Let's assume that we have two features where one feature is measured on a scale from 1 to 10 and the second feature is measured on a scale from 1 to 100,000, respectively. When we think of the squared error function in Adaline, it makes sense to say that the algorithm will mostly be busy optimizing the weights according to the larger errors in the second feature.

- **Improved Convergence in Optimization Algorithms**: Algorithms like gradient descent converge faster with feature scaling because it ensures that features contribute equally to the computation of gradients.

- **Enhanced Performance of Distance-Based Algorithms**: Methods such as K-Nearest Neighbors (KNN) and K-Means clustering rely on distance metrics; unscaled features can distort these distances, leading to inaccurate results.
- **Regularization**: When applying regularization techniques, scaling ensures that the penalty terms are applied uniformly across all features.

Common techniques

1. Normalization (min-max scaling)

Description: The simplest method is rescaling the range of features to scale the range in $[0, 1]$.
Use Case: Useful when the distribution of data is not Gaussian and when using algorithms that do not assume any distribution of the data. May not preserve the relationships between the data points. Sensitive to outliers.
Example: If a feature has values ranging from 10 to 100, normalization will scale this to a range of 0 to 1.
Forumla:

$$x_{\text{norm}}^{(i)} = \frac{x^{(i)} - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

Where,

- $x^{(i)}$ is a particular example.
- x_{min} / x_{max} is the smallest / largest value in a feature column.
- `MinMaxScaler()` is the python function.

2. Standardization (Z-score Normalization)

Description: Feature standardization makes the values of each feature in the data have zero-mean (when subtracting the mean in the numerator) and unit-variance.
Use Case: Preferred when the data follows a Gaussian distribution. Preserves the relationships between the data points. Less sensitive to outliers.
Example: A feature with a mean of 50 and a standard deviation of 5 will be transformed to have a mean of 0 and a standard deviation of 1.
Formula:

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

Where,

- μ_x is the sample mean of a particular feature column.
- σ_x is the corresponding standard deviation.
- `StandardScaler()` : Centers the data around zero, robust to outliers.

3. Robust Scaling

Description: Uses the median and interquartile range for scaling, making it robust to outliers.
Use Case: Effective when the dataset contains outliers.
Example: A feature with a median of 20 and an IQR of 10 will be scaled accordingly, reducing the influence

of outliers.

Formula:

$$x' = \frac{x - \text{median}(x)}{IQR}$$

where IQR is the interquartile range.

4. Unit Vector Scaling (Normalization)

Description: Scales the feature vector to have a unit norm, typically used in text classification and clustering.

Use Case: Useful when the direction of the data matters more than the magnitude.

Example: A vector [3, 4] will be scaled to [0.6, 0.8], preserving its direction but adjusting its magnitude to 1.

Formula:

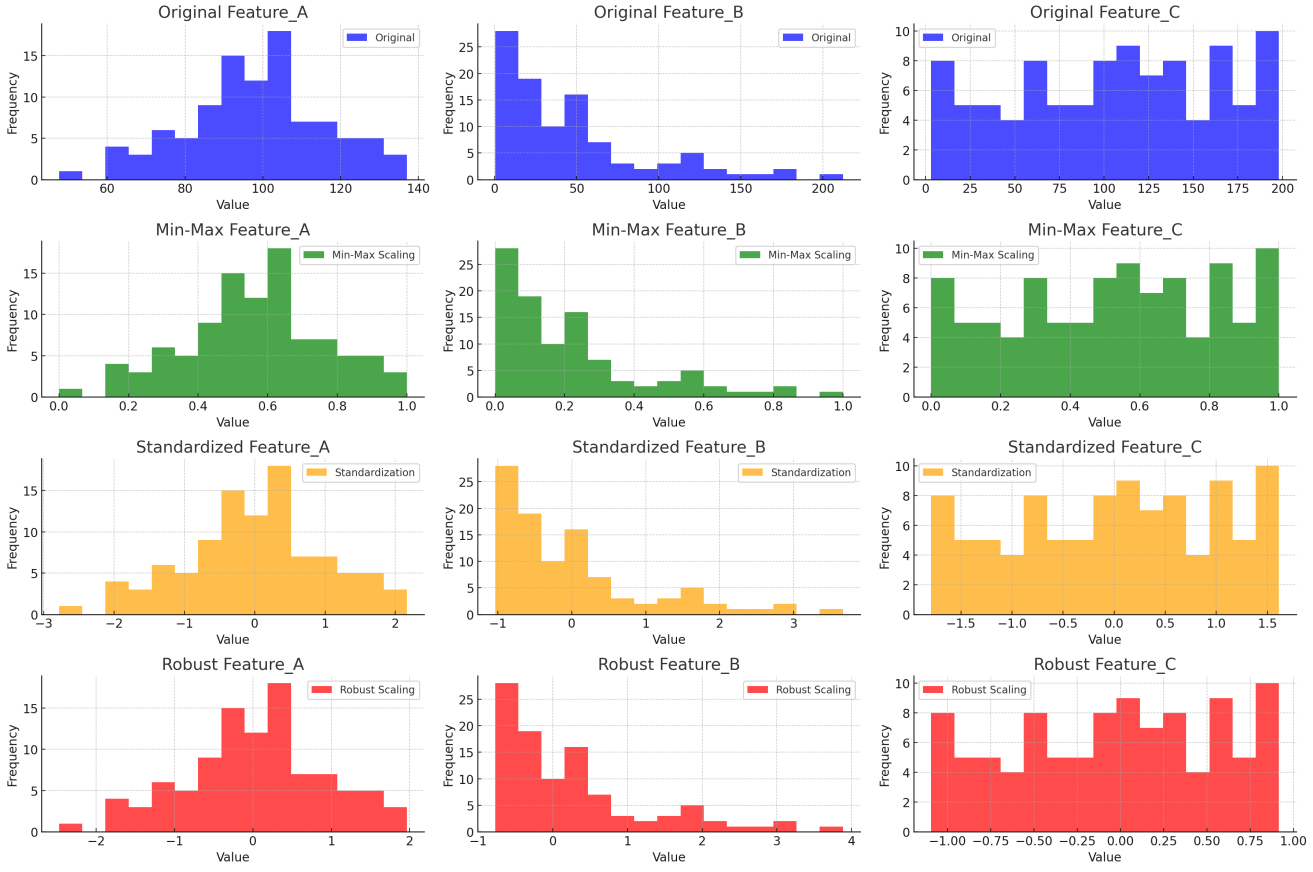
$$x' = \frac{x}{\|x\|}$$

where $\|x\|$ is the norm of the vector.

Illustration

The different feature scaling techniques on a sample dataset with three features:

1. **Feature_A**: Follows a normal distribution.
2. **Feature_B**: Follows an exponential distribution.
3. **Feature_C**: Follows a uniform distribution.



- **1st Row**: Original data distributions for Features A, B, and C.
- **2nd Row**: Min-Max Scaling applied to Features A, B, and C.
- **3rd Row**: Standardization applied to Features A, B, and C.
- **4th Row**: Robust Scaling applied to Features A, B, and C.

Feature Selection

Overview

- Reduce complexity and improve model performance
- Overcome overfitting
- Select the most informative features

Methods

Filter methods: Statistical measures
Filter type methods select features based only on general metrics like the correlation with the variable to predict. Filter methods suppress the least Filter Methods interesting variables. The other variables will be part of a classification or a regression model used to classify or to predict data. These methods are particularly effective in computation time and robust to overfitting.

- *Univariate Feature Selection*: Selects the best features based on univariate statistical tests.
  - `SelectFromModel()` : Selects the best features based on the coefficients of a machine learning model.
  - `SelectKBest()` : Selects the best k features based on univariate statistical tests.
- *Correlation*: Features should be uncorrelated with each other and highly correlated to the feature we're trying to predict.
- *Covariance*: A measure of how much two random variables change together.
- *Linear Discriminant Analysis (LDA)*
- *ANOVA: Analysis of Variance*
  - `VarianceThreshold()` : Selects features that have a minimum variance.
- *Chi-Square Test*

Wrapper methods: Search algorithm
Wrapper methods evaluate subsets of variables which allows, unlike Forward Selection filter approaches, to detect the possible interactions between Wrapper Methods variables. The two main disadvantages of these methods are : The increasing overfitting risk when the number of observations is insufficient. AND. The significant computation time when the number of variables is large.

- *Forward Selection*
- *Backward Elimination*
- *Recursive Feature Elimination (RFE)*
  - `RFE()` : Recursively eliminates features and builds a model with the remaining features until the desired number of features is reached.

- *Genetic Algorithms*

Embedded methods: Part of learning algorithm
Embedded methods try to combine the advantages of both previous methods. A learning algorithm takes advantage of its own variable selection process and performs feature selection and classification simultaneously.

- *Lasso regression* performs L1 regularization which adds penalty equivalent to absolute value of the magnitude of coefficients.
- *Ridge regression* performs L2 regularization which adds penalty equivalent to square of the magnitude of coefficients.

Regularization for feature selection

Regularization is one approach to tackling the problem of overfitting by adding additional information and thereby shrinking the parameter values of the model to induce a penalty against complexity.

The 3 popular regularization methods for linear regression:

1. L1 regularization: Least Absolute Shrinkage and Selection Operator (LASSO)
2. L2 regularization: Ridge Regression
3. Elastic Net

L1 regularization: LASSO
Depending on the regularization strength, certain weights can become zero, which encourages sparsity in weights. Useful when many features are irrelevant, as it automatically performs feature selection. Regularize loss function with LASSO:

$$L(\mathbf{w}, b)_{\text{Lasso}} = L(\mathbf{w}, b) + \lambda \|\mathbf{w}\|_1$$

L1 penalty for LASSO is defined as the sum of the absolute magnitudes of the model weights, as follows:

$$\lambda \|\mathbf{w}\|_1 = \lambda \sum_{j=1}^{m} |w_j|$$

- Increase λ → Increase regularization strength → Decrease weights of model

L2 regularization: Ridge Regression
Ridge regression is an L2 penalized model where we simply add the squared sum of the weights to the loss function. Encourages weights (w_j) to be small but does not force them to be exactly zero. Works well when most features are relevant to the task. Regularize loss function with Ridge Regression:

$$L(\mathbf{w}, b)_{\text{Ridge}} = L(\mathbf{w}, b) + \lambda \|\mathbf{w}\|_2^2$$

L2 term is defined as follows:

$$\lambda \|\mathbf{w}\|_2^2 = \lambda \sum_{j=1}^{m} w_j^2$$

- Increase λ → Increase regularization strength → Decrease weights of model

Elastic Net

Elastic Net is a compromise between ridge regression and LASSO.

It has an L1 penalty to generate sparsity and an L2 penalty such that it can be used for selecting more than n features if $m > n$:

$$L(\mathbf{w}, b)_{\text{Elastic Net}} = L(\mathbf{w}, b) + \lambda_2 \|\mathbf{w}\|_2^2 + \lambda_1 \|\mathbf{w}\|_1$$

Recall that λ, **regularization parameter**, controls the strength of the penalty.

- **Small** λ: Minimal regularization, allowing the model to closely fit the training data.
- **Large** λ: Strong regularization, simplifying the model but potentially underfitting the data.
- λ must be tuned, often using cross-validation (will be introduced in the later section).
